Chapter 11. Tree-Based Methods and Ensembles

Lecture 8 (regression and classification trees, pruning, bagging, OOB, random forests, boosting, AdaBoost, GBM, XGBoost, LightGBM, CatBoost, importance, SHAP)

Contents

1 Decision Trees **2**

2 Regression Trees **2**

2.1 Example: Hitters data . . . 2

2.2 Building a tree . . . 2

2.3 Recursive binary splitting . . . 2

2.4 Pruning . . . 3

3 Classification Trees **3**

3.1 Trees versus linear models . . . 4

3.2 Advantages and disadvantages of trees . . . 4

4 Bagging **5**

4.1 Idea . . . 5

4.2 Out-of-bag error estimation . . . 5

5 Random Forests **5**

6 Boosting **5**

6.1 Boosting regression trees . . . 5

6.2 AdaBoost . . . 6

6.3 Gradient boosting machines . . . 6

6.4 XGBoost, LightGBM, CatBoost . . . 7

7 Variable Importance **7**

7.1 Impurity-based importance . . . 7

7.2 Permutation importance . . . 7

7.3 SHAP values . . . 7

1 Decision Trees

Trees *stratify* the predictor space into simple regions, for regression and classification. They are interpretable but not very accurate; bagging, random forests and boosting combine many trees, improving accuracy at the price of interpretability.

2 Regression Trees

2.1 Example: Hitters data

Log salary from `Years` (in the major leagues) and `Hits` (previous year); two internal nodes, three leaves:

$$R_1 = \{X : \texttt{Years} < 4.5\}, \quad R_2 = \{X : \texttt{Years} \geq 4.5,\ \texttt{Hits} < 117.5\},$$

$$R_3 = \{X : \texttt{Years} \geq 4.5,\ \texttt{Hits} \geq 117.5\},$$

each leaf predicting its mean response. Regions are **terminal nodes**, split points **internal nodes**.

Interpretation. Years matters most; Hits matters only for players with five or more years: an interaction a linear model would need to be told about.

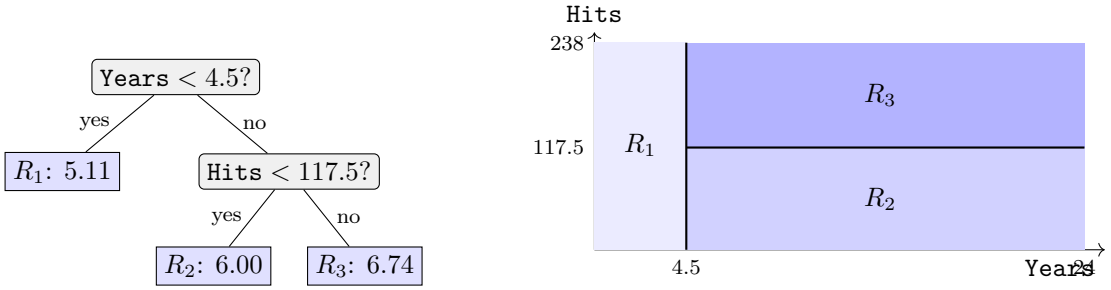


Figure 1: The Hitters tree (mean log salaries) and its partition.

2.2 Building a tree

1. Divide the predictor space into J distinct, non-overlapping regions $R_1, \ldots, R_J$.
2. For every observation in R_j, predict the mean response of the training observations in R_j.

With boxes as regions, minimize

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

$\hat{y}_{R_j}$ the mean response in box j.

2.3 Recursive binary splitting

All partitions are infeasible; use a **top-down, greedy** search (best split *at each step*, no look-ahead).

Algorithm: recursive binary splitting

Select the predictor X_j and cutpoint s that minimize

$$\sum_{i:x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2,$$

$$R_1(j,s) = \{X : X_j < s\}, \; R_2(j,s) = \{X : X_j \geq s\}.$$

Then repeat within one of the resulting regions (not the whole space), and so on, until a stopping criterion is reached (e.g. no region contains more than five observations).

The search over s is fast (sort, running sums).

2.4 Pruning

A large tree overfits. Stopping when a split gains little is short-sighted (a worthless split may precede a very good one); instead grow a large T_0 and prune.

Definition: cost-complexity (weakest-link) pruning

For each $\alpha \geq 0$ there is a subtree $T \subset T_0$ minimizing

$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|,$$

where $|T|$ is the number of terminal nodes, R_m the rectangle of the mth terminal node and $\hat{y}_{R_m}$ its mean.

$\alpha = 0$ gives T_0; as α grows, branches are pruned in a nested way: collapsing node t (branch T_t) pays off once $\alpha \geq \{R(t) - R(T_t)\}/(|T_t| - 1)$, the weakest link first.

Algorithm: building a regression tree

1. Grow a large tree by recursive binary splitting, stopping only when each terminal node has fewer than some minimum number of observations.
2. Apply cost-complexity pruning to obtain the sequence of best subtrees as a function of α.
3. Choose α by K-fold CV: for each k, repeat steps 1–2 on all but the kth fold and evaluate the MSE on the kth fold as a function of α; average and pick $\hat{\alpha}$.
4. Return the subtree from step 2 corresponding to $\hat{\alpha}$.

Hitters (132 training, 131 test): six-fold CV picks three leaves, close to the test-error minimum.

3 Classification Trees

Predict the most common class in the region (proportions as probabilities); split by an impurity instead of RSS.

Definition: node impurity measures

With $\hat{p}_{mk}$ the proportion of training observations in region m from class k:

$$E = 1 - \max_k \hat{p}_{mk} \text{ (classification error)}, \qquad G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}) \text{ (Gini)},$$

$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk} \text{ (cross-entropy)}.$$

The error is not sensitive enough for growing. Gini (sum of Bernoulli variances) is small when all $\hat{p}_{mk}$ are near 0 or 1: **node purity**. Cross-entropy is numerically similar.

Example: why Gini is more sensitive than the error rate [Supplement]

400 cases per class, split into $(300, 100)$, $(100, 300)$ or into $(200, 400)$, $(200, 0)$: both have error 0.25, but Gini $\frac{1}{2}(0.375) + \frac{1}{2}(0.375) = 0.375$ versus $\frac{3}{4}(0.444) + \frac{1}{4}(0) = 0.333$ prefers the pure node.

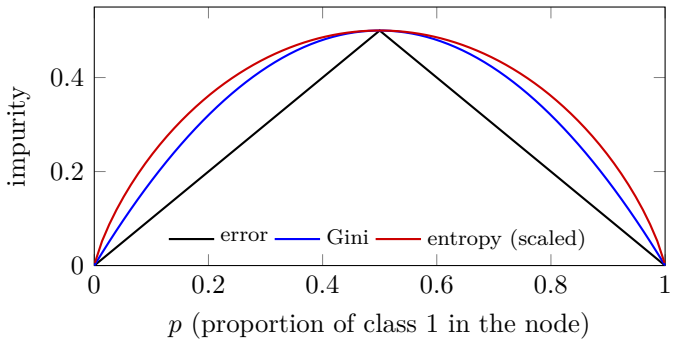


Figure 2: Two-class node impurity (entropy scaled to 0.5 at $p = \frac{1}{2}$).

Gini and entropy are strictly concave, so any split weakly decreases the weighted impurity (Jensen); the piecewise-linear error often shows no gain. For pruning, the error rate is preferred when accuracy is the goal.

Heart data: binary HD for 303 chest-pain patients, 13 predictors; CV gives six leaves. Qualitative predictors are split directly by levels.

3.1 Trees versus linear models

Linear: $f(X) = \beta_0 + \sum_j \beta_j X_j$; tree: $f(X) = \sum_{m=1}^{M} c_m \mathbb{I}\{X \in R_m\}$. Near-linear truth favors the linear model, box-like truth the tree. A circular boundary $X_1^2 + X_2^2 = 4.6$ is approximated by a staircase of boxes.

3.2 Advantages and disadvantages of trees

Easy to explain and display, close to human decisions, no dummy variables needed. But less accurate and non-robust: a small data change can change the tree a lot.

4 Bagging

4.1 Idea

Averaging reduces variance (σ^2/n for a mean). Bootstrap aggregation fits $\hat{f}^{*b}$ on B bootstrapped training sets and averages:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x).$$

Classification: **majority vote** (or averaged probabilities). Trees are grown deep, unpruned (low bias, high variance).

Theorem: variance of an average of identically distributed predictors

If $\hat{f}^{*1}, \ldots, \hat{f}^{*B}$ at a point x each have variance σ^2 and pairwise correlation ρ, then

$$\text{Var}\Big(\frac{1}{B}\sum_{b} \hat{f}^{*b}(x)\Big) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Proof [Supplement]. $\text{Var}(\sum_b \hat{f}^{*b}) = B\sigma^2 + B(B-1)\rho\sigma^2$; divide by B^2. ■

The $\rho\sigma^2$ term remains as $B \to \infty$: overlapping bootstrap samples correlate the trees. Large B does not overfit.

4.2 Out-of-bag error estimation

Theorem: each bagged tree uses about two thirds of the observations

The probability that a given observation is *not* in a bootstrap sample is $(1 - 1/n)^n \to e^{-1} \approx 0.368$.

Proof [Supplement]. Each of the n independent draws misses it with probability $1 - 1/n$; and $n \log(1 - 1/n) \to -1$. ■

Each observation is **out-of-bag** (OOB) for about $B/3$ trees; averaging their predictions gives a valid test-error estimate, for large B essentially the LOOCV error without refitting. Heart data: OOB errors track test errors; bagging and random forests beat a single tree.

5 Random Forests

Trees on bootstrap samples, but each split may use only one of m predictors sampled afresh from the p; typically $m \approx \sqrt{p}$ (classification) or $p/3$ (regression). This **decorrelates** the trees: a strong predictor no longer dominates every top split, lowering ρ. $m = p$ is bagging.

Gene-expression data: 4,718 genes, 349 patients, 15 classes; on the 500 highest-variance genes, $m < p$ slightly beats bagging; a single tree errs 45.7%. No overfitting as B grows.

6 Boosting

6.1 Boosting regression trees

Trees are grown **sequentially**, each using the previous ones; no bootstrap.

Algorithm: boosting for regression trees

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i.
2. For $b = 1, \dots, B$: fit a tree $\hat{f}^b$ with d splits ($d+1$ leaves) to the data (X, r); update $\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x)$ and the residuals $r_i \leftarrow r_i - \lambda \hat{f}^b(x_i)$.
3. Output $\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x)$.

Boosting **learns slowly**: small trees on the residuals, further slowed by λ.

Three tuning parameters.

- B: boosting can (slowly) overfit; choose by CV.
- λ: learning rate, typically 0.01 or 0.001; small λ needs large B.
- d: interaction depth; $d = 1$ (stumps) gives an additive model.

Gene-expression data: boosted stumps ($\lambda = 0.01$) slightly beat depth-2 boosting and random forests.

6.2 AdaBoost

Algorithm: AdaBoost, $y \in \{-1, +1\}$

Start with weights $D_1(i) = 1/n$. For $b = 1, \dots, B$:
(i) train a weak learner h_b using weights D_b;
(ii) compute its weighted error $\epsilon_b = \sum_i D_b(i) \mathbb{I}\{h_b(x_i) \neq y_i\}$;
(iii) set $\alpha_b = \frac{1}{2} \log \frac{1-\epsilon_b}{\epsilon_b}$;
(iv) update $D_{b+1}(i) = D_b(i) \exp\{-\alpha_b y_i h_b(x_i)\}/Z_b$, Z_b a normalizer. Output $H(x) = \operatorname{sign}\big(\sum_{b=1}^{B} \alpha_b h_b(x)\big)$.

Misclassified observations gain weight; accurate learners get a larger vote.

Theorem: AdaBoost minimizes the exponential loss stagewise

With $F_{b-1} = \sum_{s<b} \alpha_s h_s$, the pair (α_b, h_b) minimizes $\sum_i \exp\{-y_i(F_{b-1}(x_i) + \alpha h(x_i))\}$.

Proof [Supplement]. With $D_b(i) \propto e^{-y_i F_{b-1}(x_i)}$, the objective is proportional to $e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon$, which for $\alpha > 0$ is increasing in the weighted error ϵ of h (so h_b minimizes it), and setting the α-derivative to zero gives $e^{2\alpha} = (1-\epsilon)/\epsilon$. The new weights are $e^{-y_i F_b(x_i)}$, i.e. the update rule. ■

Toy example: three stumps with $\epsilon = 0.30, 0.21, 0.14$ and votes $0.42, 0.65, 0.92$.

6.3 Gradient boosting machines

Gradient descent in function space for a differentiable loss L:

(i) at step b compute pseudo-residuals $r_{ib} = -\partial L(y_i, F(x_i))/\partial F(x_i)$ at $F = F_{b-1}$;

(ii) fit a tree to them;

(iii) update $F_b = F_{b-1} + \lambda h_b$. For $\frac{1}{2}(y-F)^2$ the pseudo-residual is $y - F$: the algorithm above.

Problem	Loss	Negative gradient
regression	squared $\frac{1}{2}(y-F)^2$; absolute $\lvert y-F\rvert$; Huber	$y-F$; $\operatorname{sign}(y-F)$; clipped residual
classification	Bernoulli deviance $\log(1+e^{-2yF})$; exponential e^{-yF}	$2y/(1+e^{2yF})$; ye^{-yF}

GBM can overfit, has no explicit complexity penalty, and is sequential.

6.4 XGBoost, LightGBM, CatBoost

XGBoost: regularized objective

$$\mathcal{L} = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{b=1}^{B} \Omega(f_b), \qquad \Omega(f_b) = \gamma T_b + \frac{\lambda}{2} \sum_{j=1}^{T_b} w_{bj}^2,$$

T_b leaves with scores w_{bj}. A second-order expansion (gradients g_i, Hessians h_i) gives $w_j^* = -\sum_{i \in I_j} g_i / (\sum_{i \in I_j} h_i + \lambda)$ and a closed-form split gain; plus parallel, cache-aware computation.

LightGBM: histogram-based split finding and leaf-wise growth for large, high-dimensional data.

CatBoost: ordered target statistics for categorical features and *ordered boosting* against target leakage and prediction shift.

Method	Main focus	Mechanism
XGBoost	regularization and second-order optimization	controls tree complexity; uses gradients and Hessians
LightGBM	speed and scalability	histogram-based splitting, leaf-wise growth
CatBoost	categorical data, bias reduction	ordered target statistics and ordered boosting

7 Variable Importance

7.1 Impurity-based importance

$$\mathrm{Imp}(X_j) = \sum_{\text{splits on } X_j} \text{reduction in loss}, \qquad \text{final importance} = \frac{1}{B} \sum_{b=1}^{B} \mathrm{Imp}_b(X_j),$$

with RSS, Gini or cross-entropy. Heart data: Thal, Ca and ChestPain lead. A global, non-causal view.

Limitations: biased toward variables with many possible splits; model-dependent; not comparable across models.

7.2 Permutation importance

Shuffle X_j and measure the error increase:

$$\mathrm{Imp}(X_j) = \mathrm{Error}_{\mathrm{perm}(j)} - \mathrm{Error}_{\mathrm{original}}.$$

Model-agnostic, on OOB or validation data; a correlated substitute can mask importance.

7.3 SHAP values

Shapley additive explanations:

$$f(x) = \phi_0 + \sum_{j=1}^{p} \phi_j, \qquad \phi_j = \sum_{S \subseteq \{1,\ldots,p\} \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \big[v(S \cup \{j\}) - v(S) \big],$$

$v(S) = \mathbb{E}[f(X) \mid X_S = x_S]$, $\phi_0 = \mathbb{E} f(X)$; ϕ_j is the average marginal contribution of X_j over all feature orders. The unique attribution with efficiency ($\sum_j \phi_j = f(x) - \phi_0$), symmetry, dummy and additivity; *local* per observation (TreeSHAP for trees).

Method	Type	Characteristic
tree importance	model-specific	fast but biased
permutation	model-agnostic	based on prediction error
SHAP	model-agnostic	local and consistent explanations

Summary

- Trees: recursive binary splitting, cost-complexity pruning with CV.
- Bagging averages deep bootstrap trees (OOB error); random forests decorrelate them.
- Boosting fits small trees to residuals with shrinkage (B, λ, d); AdaBoost: exponential loss; GBM: negative gradient.